



## REDES NEURONALES Y APRENDIZAJE PROFUNDO

Nombres: MACÍAS MONDRAGÓN MARCOS RODLFO, REYES CRUZ ROBERTO MISAEEL

Especialidad: Inteligencia Artificial

Fecha de la práctica: 24 de mayo de 2026

**Nombre de la práctica: Clasificación de imágenes utilizando redes convolucionales en TensorFlow.**

### Introducción:

Las redes neuronales convolucionales (CNN) son un tipo de modelo de aprendizaje profundo diseñado para procesar datos con una estructura de cuadrícula, como imágenes. En esta práctica, implementaremos una CNN en TensorFlow para clasificar imágenes del conjunto de datos **CIFAR-10**, que contiene imágenes de 10 categorías diferentes.

### Objetivos:

1. Comprender la arquitectura básica de una CNN.
2. Implementar una CNN utilizando TensorFlow y Keras.
3. Entrenar y evaluar una CNN para la clasificación de imágenes.
4. Visualizar los resultados y la precisión del modelo.

### Requisitos previos:

1. Conocimientos básicos de Python.
2. Tener instalado TensorFlow en tu máquina.

### Procedimiento:

#### Paso 1: Importar librerías y cargar el conjunto de datos

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
import matplotlib.pyplot as plt

# Cargar datos de CIFAR-10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalizar los datos
x_train = x_train / 255.0
x_test = x_test / 255.0

# Convertir etiquetas a una representación categórica
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

## Paso 2: Explorar los datos

Visualizar algunas imágenes del conjunto de datos.

```
# Mostrar las primeras 10 imágenes de entrenamiento
class_names = ['Avión', 'Automóvil', 'Pájaro', 'Gato', 'Ciervo', 'Perro', 'Rana',
               'Caballo', 'Barco', 'Camión']
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[i])
    plt.axis('off')
plt.show()
```

## Paso 3: Definir la arquitectura de la CNN

Crear una CNN básica con capas convolucionales, de agrupación (pooling) y densas.

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary() # Ver la arquitectura del modelo
```

## Paso 4: Compilar y entrenar el modelo

Configurar el modelo y entrenarlo.

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=10,
                   validation_data=(x_test, y_test))
```

## Paso 5: Evaluar el modelo

Observar el desempeño del modelo y graficar los resultados.

```
# Evaluar el modelo
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Precisión en el conjunto de prueba: {test_acc * 100:.2f}%")

# Graficar la pérdida y la precisión
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Precisión entrenamiento')
plt.plot(history.history['val_accuracy'], label='Precisión validación')
plt.legend()
plt.title('Precisión')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Pérdida entrenamiento')
```

```
plt.plot(history.history['val_loss'], label='Pérdida validación')
plt.legend()
plt.title('Pérdida')
plt.show()
```

### **Preguntas para los estudiantes:**

1. ¿Qué función cumplen las capas convolucionales en la CNN?
2. ¿Por qué normalizamos los datos antes de entrenar el modelo?
3. ¿Qué sucede si aumentamos o reducimos el número de filtros en las capas convolucionales?
4. Analiza las gráficas de pérdida y precisión: ¿Cómo podemos mejorar el modelo?

### **Conclusiones:**

- Discute cómo las CNN procesan imágenes y su capacidad para extraer características importantes.
- Reflexiona sobre los resultados obtenidos y las posibles mejoras al modelo.
- Explica la importancia de ajustar hiperparámetros y utilizar regularización para evitar el sobreajuste.

### **Respondiendo:**

1. ¿Qué función cumplen las capas convolucionales en la CNN?  
Estas son las encargadas de extraer características importantes de las imágenes. Aplican filtros o kernels sobre la imagen para detectar los patrones como bordes, texturas, y demás características relevantes. En las primeras capas se identifican características sencillas, como líneas y contornos. Y conforme avanza, las capas profundas detectan patrones complejos, como partes de objetos u objetos completos.
2. ¿Por qué normalizamos los datos antes de entrenar el modelo?  
La normalización se realiza para escalar los valores de los píxeles a un rango pequeño, generalmente entre 0 y 1. En este caso, los datos se dividen entre 255 porque los píxeles originales tienen valores entre 0 y 255.
3. ¿Qué sucede si aumentamos o reducimos el número de filtros en las capas convolucionales?  
Si aumentamos el número de filtros:
  - o La red puede aprender más características y patrones complejos.
  - o El modelo suele tener mayor capacidad de aprendizaje.
  - o Incrementa el tiempo de entrenamiento y el uso de memoria.
  - o Existe mayor riesgo de sobreajuste si el modelo se vuelve demasiado complejo.Si reducimos el número de filtros:
  - o El modelo será más rápido y ligero.
  - o Disminuye el consumo de recursos computacionales.
  - o Puede perder capacidad para detectar características importantes.
  - o La precisión podría disminuir debido a una menor capacidad de representación.

4. Analiza las gráficas de pérdida y precisión: ¿Cómo podemos mejorar el modelo?

Las grafica de precisión muestra que el entrenamiento aumenta constantemente hasta aproximadamente 77%, mientras que la validación alcanza cerca de 71%, con una fluctuación en la tercera época. Esto indica que el modelo sí está aprendiendo correctamente los patrones de las imágenes.

En la gráfica de pérdida, el entrenamiento disminuye continuamente, mientras que la pérdida de validación también baja, aunque presenta una fluctuación (al igual que el de precisión). Esto sugiere que el modelo comienza a presentar un ligero sobreajuste en la tercera época, ya que el rendimiento en entrenamiento sigue mejorando más que en validación.

Para mejorar en el modelo, se pueden aplicar:

- o Incrementar el número de épocas de entrenamiento de forma controlada.
- o Utilizar técnicas de regularización como Dropout.
- o Aplicar Data Augmentation para generar más variaciones de imágenes.
- o Ajustar la cantidad de filtros y capas convolucionales.
- o Modificar hiperparámetros como la tasa de aprendizaje.
- o Implementar Early Stopping para detener el entrenamiento cuando la validación deje de mejorar.

## Conclusiones

Las redes neuronales convolucionales (CNN) son modelos especializados en el procesamiento de imágenes, ya que utilizan capas convolucionales capaces de detectar automáticamente características importantes como bordes, texturas, formas y patrones. A medida que la información avanza por la red, las características extraídas se vuelven más complejas, permitiendo identificar objetos completos dentro de las imágenes. Gracias a esto, las CNN logran un excelente desempeño en tareas de clasificación de imágenes.

En los resultados obtenidos durante la práctica, se observó que el modelo alcanzó una buena precisión tanto en entrenamiento como en validación, demostrando que la red logró aprender correctamente las características del conjunto de datos CIFAR-10. Sin embargo, también se aprecia una pequeña diferencia entre ambas precisiones, lo cual puede indicar un inicio de sobreajuste. Para mejorar el desempeño del modelo podrían agregarse más capas convolucionales, aumentar la cantidad de datos mediante Data Augmentation, entrenar durante más épocas o utilizar arquitecturas más avanzadas.

Además, esta práctica permitió comprender la importancia de ajustar adecuadamente los hiperparámetros, como la tasa de aprendizaje, el número de filtros, el tamaño del batch y la cantidad de épocas. Estos parámetros influyen directamente en la capacidad de aprendizaje y generalización del modelo. Asimismo, el uso de técnicas de regularización, como Dropout o Early Stopping, es fundamental para evitar el sobreajuste, logrando que la red neuronal no memorice los datos de entrenamiento y pueda funcionar correctamente con imágenes nuevas.